

Autoresearch Loops

A Causal Study of the Instruction File

Christian Block

Research project with Prof. Ramesh

September 7, 2026

An autoresearch loop runs its own experiments

- The model is given a goal, code it may edit, and an executor
- The executor runs one trial and returns a single scalar
- From there it proceeds without a human in the loop

The cycle

Propose a change, execute it, read the outcome, decide what to attempt next, repeat until the budget is exhausted.

- Progress is enforced by version control: commit, train, keep if the score improved, otherwise `git reset`
- The repository only ever advances

Why this matters now

- Karpathy's autoresearch, March 2026
- Forked several thousand times since
- It is what fixed the current meaning of the term
- People are running it against production systems

Shopify Liquid

About 120 experiments

53% faster parse and render

Author's own verdict: probably overfitted

Never merged

The uncomfortable part

The loops demonstrably function. The reasons for their behaviour are unknown, including to the people running them.

The architecture is three files, and one of them is the controller

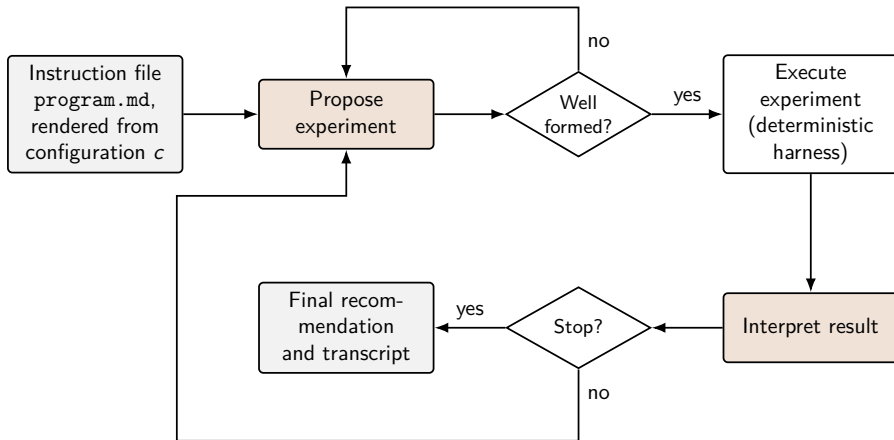
File	Holds	Edited by
<code>prepare.py</code>	the data and the score	nobody
<code>train.py</code>	the model and the training loop	the agent
<code>program.md</code>	the instructions	the human

- `prepare.py` is unreachable, so every experiment is scored identically
- No scheduler, no driver program, no orchestration code of any kind
- The nine step experiment cycle exists only as English prose inside `program.md`, and the model itself is the execution layer

The observation the project rests on

`program.md` is not configuration supplied to a controller. It *is* the controller. A controller written in code can be read. This one cannot.

Figure 1: control flow of one run



Shaded boxes are language model calls. The execute box is deterministic, so every number the loop reasons about was measured and not generated. A rejected proposal returns to the proposal step without spending an experiment.

Research questions

RQ1

Which components of `program.md` causally determine which aspects of the loop's research *process*, and how large is each effect?

RQ2

Do the components act independently of one another, or does the effect of one depend on the level of another?

- The contribution is a method, not an architecture. The aim is not a better loop, but an account of what an existing one responds to
- Scope: no human baseline and no external standard of good practice. Every comparison is between two configurations of the same system

From prose to a configuration vector

- One narrowing first: the search space is a hyperparameter space, not arbitrary code, so the instruction file stays the only free variable
- Every file is rendered from one template
- The fixed part gives the research question, the executor interface and the JSON response format, byte identical across all 32 files
- The variable part is five paragraphs, each written at one of two levels. Nothing else in the file varies

A file becomes a point in a space

$$c = (c_1, \dots, c_5), \quad c_i \in \{0, 1\} \quad \implies \quad 2^5 = 32 \text{ files}$$

The five components: the independent variables

	Component	Level 0	Level 1
<i>M</i>	Evaluation criterion	Success left undefined; the loop judges quality as it sees fit.	Success is 5 fold cross validated accuracy; the standard deviation breaks ties only.
<i>B</i>	Search breadth	Commit to one model family for the session.	Compare both families and vary several hyperparameters at once.
<i>S</i>	Stopping rule	Fixed budget of exactly three experiments.	Adaptive budget up to eight, halt when improvement falls below 0.002.
<i>O</i>	Reporting style	Terse: the action taken and the numbers involved.	Full sentences: hypothesis, expectation, interpretation.
<i>E</i>	Search emphasis	Explore first: several configurations before refining any.	Exploit first: refine a promising configuration immediately.

These categories were not invented for the study. A production `program.md` hardcodes a baseline to exceed, specifies failure handling, fixes a reporting discipline and imposes a parsimony rule.

Every file is rendered from this template

```
# Research Task: {{ task_title }}                                <- set by the task, not by c

You are an autonomous research agent. Your task is to investigate the following
question through real, executed experiments (not simulated ones):

> {{ research_question }}                                       <- set by the task, not by c

You have access to a fixed dataset ({{ dataset_name }}, already loaded and split
into train/validation) and to a tool that fits either a RandomForestClassifier or
a GradientBoostingClassifier with hyperparameters you choose. The tool always
returns the same record: 5-fold cross-validated accuracy on the training split,
its standard deviation across folds, and accuracy on the held-out validation split.

## Evaluation
{{ metric_text }}                                              <== slot M

## Search strategy
{{ breadth_text }}                                           <== slot B
{{ emphasis_text }}                                           <== slot E

## Budget and stopping
{{ stopping_text }}                                           <== slot S

## Reporting style
{{ output_format_text }}                                       <== slot 0

## Required response format
At every step, respond with a single JSON object and nothing else, matching one of these two shapes:
{"action": "propose", "hypothesis": ..., "model": "random_forest" | "gradient_boosting", "params": {...}}
{"action": "interpret", "interpretation": ..., "decision": "continue" | "stop", "final_recommendation": ...}
```

Everything outside the five marked slots is byte identical across all 32 files of a task. The three remaining substitutions are set by the dataset, so they are constant within a task and differ only between tasks.

The ten paragraphs, verbatim (1 of 2)

slot M Evaluation criterion

- [0] Evaluate model quality in whatever way you judge appropriate.
- [1] Evaluate every model using 5-fold cross-validated accuracy on the training split, AND report the standard deviation across folds as a measure of stability. A configuration is only better than another if its mean accuracy is higher; use the standard deviation only to break ties or to flag instability.

slot B Search breadth

- [0] Stay within a single model family per experiment. Pick either RandomForestClassifier or GradientBoostingClassifier for the whole session unless you have exhausted reasonable hyperparameters for it and explicitly decide to switch.
- [1] You are free to compare RandomForestClassifier and GradientBoostingClassifier side by side within the same session, and to vary multiple hyperparameters of each at once.

slot S Stopping rule

- [0] You have a fixed budget of exactly 3 experiments in total. Run exactly 3, then stop and report your final recommendation, regardless of the outcome of experiment 3.
- [1] You have a budget of up to 8 experiments. After each experiment, decide whether to continue: stop as soon as an experiment fails to improve the evaluation metric (defined above) by more than 0.002 over the best result so far, or when you reach 8 experiments, whichever comes first.

The ten paragraphs, verbatim (2 of 2)

slot 0 Reporting style

- [0] Keep every response short: state the action you are taking and the numbers involved, with minimal prose.
- [1] Explain your reasoning at every step in full sentences: what you hypothesize, why, what you expect to observe, and how you interpret the actual result relative to that expectation.

slot E Search emphasis

- [0] Prioritize breadth of search: try several meaningfully different configurations before you spend more than one experiment refining any single one of them.
- [1] Prioritize depth: as soon as one configuration looks promising, immediately refine it with nearby variations (e.g. slightly different hyperparameter values) before trying anything unrelated.

What a configuration is, concretely

$c = (1, 0, 1, 1, 0)$ is the file M1-B0-S1-01-E0: it takes paragraph [1] in the evaluation slot, [0] in breadth, [1] in stopping, [1] in reporting and [0] in emphasis. Its neighbour M1-B0-S1-00-E0 is the same file with one paragraph swapped and nothing else changed. The other 30 files are the remaining combinations.

The complete factorial makes interactions estimable

- With all 32 files, every pairwise interaction β_{ij} is estimated separately from every main effect
- The independence of the components therefore becomes a question answerable *from the data*, rather than one the design has to assume

This is what makes RQ2 testable

Does the effect of one component depend on the level of another, or do they simply add?

Tasks, executor and controls

- Breast Cancer Wisconsin, 569×30 , two classes
- Wine, 178×13 , three classes
- One stratified 70/30 split per task, same seed throughout
- Two model families under a fixed hyperparameter whitelist
- Baseline $a_0(d)$: a default fit, better of the two families, computed with no language model. 0.960 on both tasks

Held fixed, so they explain nothing

agent model pinned to gpt-4o-mini-2024-07-18, the split and its seed, the executor and whitelist, the fixed template, the response format, the harness caps. Runs are separate conversations.

Temperature = 0.7, not 0

Run to run variance *is* the error term every coefficient is tested against. Greedy decoding would drive it to zero and every standard error with it.

The executor returns the same record every time: cross validated accuracy, its across fold standard deviation, held out validation accuracy. The instruction file alters what the loop attends to, not what is measured. Every run passes an explicit sampling seed and every cell draws from the same seed list, so the draws cannot favour one configuration.

$$N = \underbrace{2^5}_{\text{configurations}} \times \underbrace{D}_{2 \text{ tasks}} \times \underbrace{R}_{20 \text{ replicates}} = 1,280 \text{ runs, 64 cells, 20 each}$$

- The complete 2^5 factorial, crossed with the task suite and replicated. Every cell holds exactly 20 completed runs, so balance is exact rather than approximate
- $R = 20$ was not chosen for convenience. It fixes the smallest detectable level switch at 0.22σ per dataset, where σ is the run to run standard deviation of the metric being fitted
- Of the 1,280 runs, 1,278 terminated on the model's own stop decision and 2 on the harness cap, so the cap is a safety net rather than a constraint that could be confounded with the stopping rule

The dependent variables: what the run achieved

Gain over default

The difference between the score of the configuration the run finally recommends and the baseline $a_0(d)$. It answers the plainest question one can ask of an autoresearch run: did the tuning achieve anything that untuned defaults would not have given for free. A negative value is not a rounding artefact but a real failure mode, a run that talked itself into a configuration worse than the one it started from.

Improvement rate

The proportion of executed trials whose score is strictly higher than the best score seen earlier in the same run. Where the gain asks what the loop ended with, this asks whether the search was learning as it went. A run of eight trials containing a single improvement is close to guessing; a high rate is evidence that the interpret step is genuinely informing the next proposal rather than decorating it.

The dependent variables: what the search cost

Wasted trial ratio

The proposals rejected before execution, whether malformed or outside the whitelist, together with those that exactly duplicate an earlier proposal in the same run, divided by all proposals attempted. Both failure modes consume a turn and return no measurement. A high ratio means the loop is either failing to satisfy the response format or failing to consult its own transcript.

Cost to best

The number of executor calls up to and including the one that produced the run's highest score, taking the earliest such call on ties. Exploration after that point does not count. Because every executed trial consumes the same fixed budget, this is simply the computation spent to reach the result.

The first two are outcome metrics, the second two efficiency metrics, and the results turn on that distinction. All four are read straight off the structured transcript: each is a difference in accuracy, a proportion of proposals, or a count of executor calls, so no judge model or scoring library enters the measurement and nothing in it can drift with the instruction being tested.

The measurement window

- The stopping rule decides how many experiments a run performs, and every metric above aggregates over those experiments
- A best score improves with the number of trials for reasons that have nothing to do with the quality of the search, and a ratio computed over more proposals is estimated more precisely
- Left uncorrected, the stopping rule would inflate its own apparent effect on any metric that accumulates over trials

Resolution

All four metrics are computed over the **first three experiments**, that being the number performed at the fixed budget level and hence the number available in every run. Complete run values are reported separately as a robustness check.

The regression model

Orthogonal coding, then main effects with the task controlled

$$X_i = +1 \text{ at level 1, } X_i = -1 \text{ at level 0}$$

$$Y = \beta_0 + \sum_{i=1}^5 \beta_i X_i + \sum_{j=2}^D \gamma_j D_j + \varepsilon$$

Interactions, free because all 32 cells were run

$$Y = \beta_0 + \sum_i \beta_i X_i + \sum_{i < j} \beta_{ij} X_i X_j + \sum_j \gamma_j D_j + \varepsilon$$

- $2\beta_i$ is the average change in Y from switching component i , and is what is reported throughout
- γ_j absorbs task difficulty and headroom above a_0 , which would otherwise inflate every standard error
- OLS on individual runs, HC3 standard errors, Benjamini and Hochberg adjustment within each metric

Main effect estimates $2\hat{\beta}_i$

Metric	M	B	S	O	E	$\hat{\gamma}$	R^2
Gain over default	+0.001	-0.001	+0.001**	+0.000	+0.000	-0.001***	0.029
Wasted trial ratio	-0.023***	-0.034***	+0.044***	-0.075***	+0.005	-0.003	0.159
Improvement rate	+0.008	-0.026*	+0.034**	+0.006	+0.018	-0.064***	0.044
Cost to best	+0.06	-0.02	+0.05	-0.01	-0.11	-0.20***	0.025

- Reporting style is the largest effect anywhere in the design, -0.075 , cutting the wasted trial ratio from 0.106 to 0.031
- The adaptive stopping rule works the other way, $+0.044$: it raises the mean number of experiments from 2.88 to 4.57, and some of the extra proposals duplicate earlier ones
- The stopping rule is the only component touching either outcome metric, and nothing survives on cost to best
- On two metrics the dataset effect $\hat{\gamma}$ is about twice the largest component effect, both at $p < 10^{-5}$. That is exactly the variance γ_j takes out of the residual

HC3 errors, Benjamini and Hochberg adjusted p . $\hat{\gamma}$ is Wine relative to Breast Cancer. Stars: adjusted $p < 0.05$, < 0.01 , < 0.001 .

What the design lets us say beyond the point estimates

RQ2: interactions

Three of forty survive adjustment, all on the wasted trial ratio, all involving reporting style. Only $S \times O$ at -0.031 is main effect sized, and its sign means *substitution*, not reinforcement. The rest are additive.

Compliance, per component

B families proposed: 0.66 / 0.82
 S experiments performed: 0.89 / 0.20
 E second proposal refines first: 0.88 / 0.69
 O prose characters: 205 / 386
 M leaves no behavioural trace

Why compliance is reported at all

It separates a component with no effect from one the model disregarded. The adaptive rule is followed in only 20% of runs, so its estimate is the effect of *offering* an adaptive budget, not of a loop that terminates on diminishing returns. All estimates are intention to treat, attenuated towards zero in proportion to non compliance.

Implications, and why outcome barely moved

- Within the range of wordings tested, the instruction file determines the **efficiency** of the search, not the **quality** of the configuration returned
- The highest leverage sentence in the file is the one about reporting style: requiring the model to state its reasoning cuts malformed and duplicate proposals by about two thirds, and costs only tokens
- An end to end score cannot attribute. A sliced file plus a full factorial can, and is not much more expensive

The ceiling binds, not the loop

Untuned defaults already reach $a_0 = 0.960$ on both tasks. 40.0% of runs improve on that and 20.9% end below it. The measured effects concern the efficiency with which a ceiling is approached, and say nothing about the height of that ceiling.

Boundary conditions

- ① **One agent model.** What was measured is the action of instructions on a single policy, not a property of instructions in general
- ② **One wording per level.** Semantics were held fixed, phrasing was not varied, so variance due to wording cannot be separated from variance due to the architectural constraint. The most serious of the four
- ③ **Two datasets.** The dataset fixed effect rests on a single contrast, and both tasks sit near the accuracy ceiling
- ④ **A hyperparameter space, not code.** The instruction file carries less of the system's behaviour than it would in Karpathy's loop

The two extensions these open

Sample several wordings per level, which separates a component from its phrasing. Then apply the procedure to a loop that edits code, where `program.md` carries considerably more of the behaviour.

Conclusion

- `program.md` does what controller code does in conventional software, yet cannot be read the way code is read, so it was treated as a configuration vector
- All $2^5 = 32$ files, two datasets, twenty replicates, 1,280 runs against a real executor
- Because the configurations were constructed rather than collected, the coefficients admit a causal reading. Because the factorial is complete, independence could be tested rather than assumed
- **RQ1:** the components determine the process, not the outcome, and reporting style dominates
- **RQ2:** three of forty interactions survive, all involving reporting style, so the components are close to additive

For tasks of this difficulty, `program.md` determines how the loop conducts its search rather than what the search returns.

Thank you. Questions welcome.